

v1.0 - TD - Technical Document - 08.10.2026

Versão: 1.0 **Data:** 08/10/2026

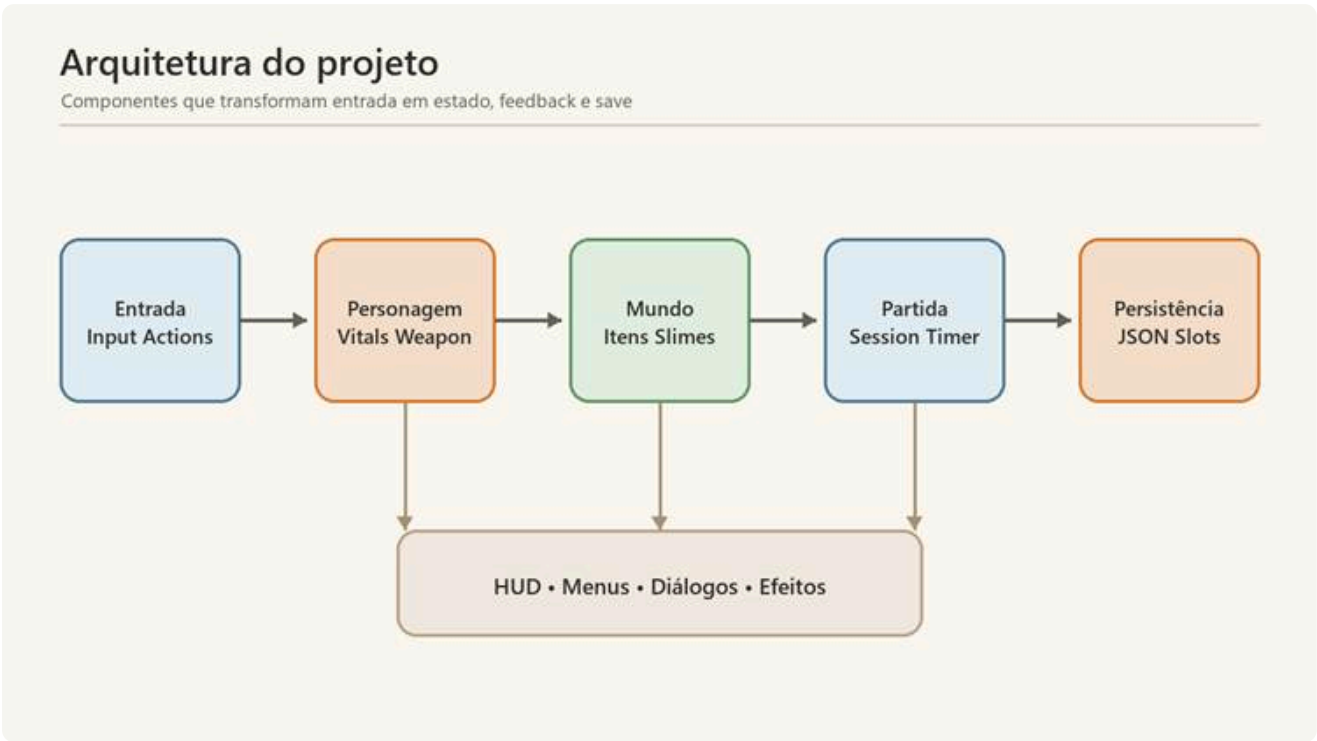
Leitor: autor do jogo e colaboradores futuros

Base: scripts autorais, cenas ativas, prefabs, Input Actions e configurações locais

Este documento explica como o jogo atual funciona e onde cada regra mora. Ele também separa as estruturas prontas das funcionalidades ainda planejadas. Para estudar, acompanhe primeiro o fluxo de uma nova partida, depois a passagem de um item do chão ao inventário, o caminho de um ataque até o dano e, por fim, o save. Os trechos em **Ponto de atenção** registram dependências reais do código; não são erros afirmados sem execução no Editor.

1 Visão de arquitetura

O projeto usa componentes MonoBehaviour ligados a GameObjects das cenas. Dados ajustáveis aparecem em campos serializados no Inspector. Prefabs fornecem inimigos, armas, itens e UI. Alguns sistemas conversam por eventos C#; outros consultam o objeto necessário na cena com FindAnyObjectByType. O estado que precisa atravessar cenas fica temporariamente em GameSession e, de forma persistente, no JSON escrito por SaveController.



Arquitetura dos sistemas

Camada	Componentes centrais	Responsabilidade
Entrada	Player Controls.inputactions, PlayerController, ActiveWeapon, PowerController, BackpackController	Traduzir teclas e mouse em movimento, ataque e interface

Camada	Componentes centrais	Responsabilidade
Regras do personagem	PlayerVitals, EquipmentController, EquipmentSlot	Vida, defesa, XP, bônus e equipamentos
Itens e UI	Item, ItemStack, InventoryController, ItemDragHandler, MenuController	Coleta, pilhas, slots, arrastar e consumir
Mundo	EnemyAI, EnemyPathfinding, EnemyHealth, SurvivalSpawner, ScenePortal	Inimigos, surgimento, colisão e troca de cena
Partida	GameSession, PrologueChallenge, DeathScreenController	Dificuldade, tempo, vitória, derrota e reinício
Persistência	SaveController, SaveData, SaveSlotService, ItemDictionary, WorldProgressKey	Converter estado vivo em JSON e restaurá-lo
Apresentação	HUDs, barras, menus, diálogos, efeitos, áudio	Mostrar o estado e o feedback ao jogador

1.1 Motor e dependências

O projeto abre com **Unity 6000.6.3f1**. Packages/manifest.json inclui Input System 1.20.0, Universal Render Pipeline 17.6.0, Unity UI 2.6.0, TextMesh Pro por dependência de UI e pacotes de tilemap/2D. A cena não depende de servidor nem de banco remoto. O save usa JsonUtility e Application.persistentDataPath. As cenas habilitadas em EditorBuildSettings.asset são, nesta ordem, MainMenu, CasaInterior e SampleScene. SampleScene_Backup_Equipamentos.unity existe como backup, fora do fluxo do build.

1.2 Convenção de leitura dos scripts

- **Awake:** cria ou encontra referências e configura o estado inicial do componente.
- **OnEnable e OnDisable:** ativam/desativam Input Actions e assinam/cancelam eventos, quando aplicável.
- **Start:** conecta objetos que já passaram pelo próprio Awake.
- **Update:** lógica por quadro, entrada, cronômetro e decisões de IA.
- **FixedUpdate:** deslocamento por Rigidbody2D e física.
- **LateUpdate:** visualização que precisa ocorrer depois da movimentação e das mudanças do quadro.
- **OnDestroy:** remove assinaturas e libera recursos de tela capturada.

2 Hierarquia e fluxo das cenas

Mapa das cenas

Relações de navegação e conteúdo • esquema sem escala



Mapa esquemático do fluxo

Cena	Objetos e componentes relevantes	Papel
MainMenu	Canvas, OnionMenuController, SaveSlotsPanel, SaveSlotPanelController, configurações	Escolher dificuldade, nome e vaga; continuar; vídeo; sair
CasaInterior	Player, câmera, Grid/Teto/interior, Madrasta, CasaSpawn, PortaParaMapa, UI	Início e reinício da tentativa; relógio desligado
SampleScene	Player, câmera, Grid/Grass/Foreground/Cosmetic, Enemies, três NPCs, PortaParaCasa, UI	Sobrevivência, tempo, geração de slimes e cura

O player aparece nas duas cenas de jogo. `ScenePortal.OnTriggerEnter2D` salva antes de chamar `SceneManager.LoadScene`. Ele guarda em `GameSession` a identificação do ponto de chegada. O `SaveController` da cena carregada lê o JSON, aplica o progresso salvo, busca o `SceneSpawnPoint` de destino e move o player para esse ponto. Há um intervalo anti repetição de 0,75 segundo baseado em `Time.unscaledTime` para evitar uma sequência imediata de portais.

Ponto de atenção: a ligação depende de `destinationScene`, `destinationSpawnId` e `spawnId` estarem escritos exatamente como nas cenas. Alterar o nome de cena ou de âncora no Inspector exige atualizar os portais.

3 Fluxo de uma nova partida

1. `OnionMenuController.OpenNewGame` abre `SaveSlotPanelController`.
2. Quatro dos botões existentes apresentam Fácil, Médio, Difícil e Insano. Depois da escolha, os cinco botões voltam a representar vagas.

3. O nome é validado: texto não vazio, até 40 caracteres. Uma vaga ocupada abre confirmação de sobrescrita.
4. `GameSession.ChooseNewGame` guarda temporariamente vaga, nome e dificuldade. `SceneManager.LoadScene` abre `CasaInterior`.
5. `SaveController.Start` encontra player, inventário, mochila e equipamentos, consome o modo `NewGame`, zera progresso, coloca o personagem no `CasaSpawn` e escreve o primeiro JSON.
6. `PrologueChallenge` cria o painel do relógio, mas sua propriedade `countTimeInThisScene` é falsa na casa. O tempo começa quando a cena externa entra em execução.

No caso de **Continuar**, o menu lê o arquivo da vaga com `SaveSlotService.TryRead` e abre a cena gravada em `SaveData.sceneName`. `SaveController.LoadGame` reconstrói o estado. No caso de **Renascer/Recomeçar**, `GameSession.ChooseRestart` mantém nome e dificuldade, e `SaveController.ResetProgress` remove o progresso da tentativa ao voltar para a casa.

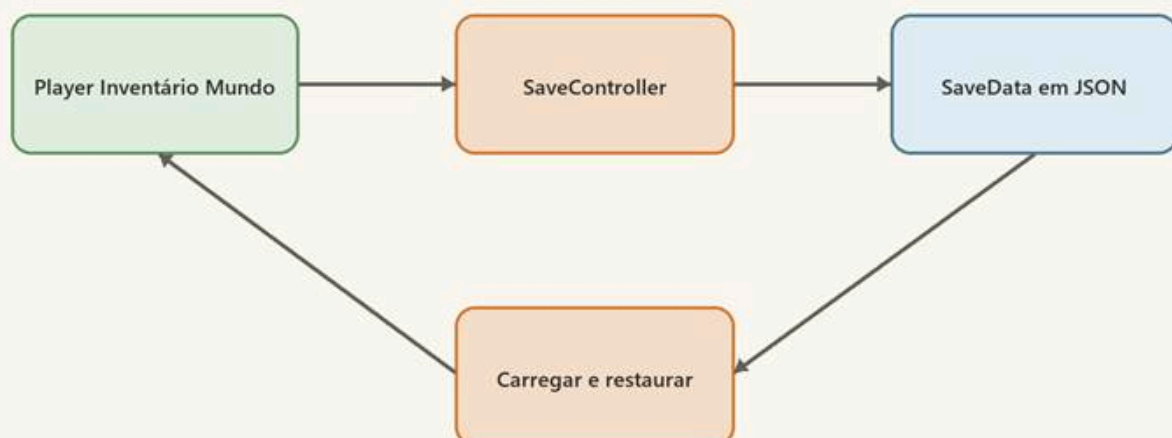
3.1 Dados de sessão e dados persistentes

`GameSession` é estática e funciona como envelope entre cenas: modo pendente (`NewGame`, `Continue`, `SceneTransition`, `Restart`), vaga, nome, âncora, caminho do save, dificuldade, tempo decorrido, vitória, bitmask de presentes e instante da próxima reposição de flechas. Ela é reiniciada na inicialização do subsistema, mas não substitui o arquivo JSON.

`SaveData` guarda os campos persistentes. `SaveController` faz a leitura e escrita com `JsonUtility`. Vagas 1 a 5 usam `save_slot_N.json` dentro de `Application.persistentDataPath`. Para compatibilidade, a vaga 1 pode ler o arquivo antigo `saveData.json` se ainda não houver seu arquivo novo. Saves antigos sem campo `difficulty` são interpretados como Médio, porque o valor padrão 0 representa Médio no enum.

Ciclo de persistência

O estado em cena é fotografado em JSON e reconstruído ao carregar



Campo persistido	Origem em execução	Destino ao carregar
Cena e posição	SceneManager e Player.transform	Cena escolhida e posição/âncora
Nome, vida, estamina, nível, XP	PlayerVitals	RestoreSavedVitals e SetPlayerName
Dificuldade e tempo	GameSession	SetDifficulty e RestoreRunProgress
Inventário, equipamentos, mochila	Controladores de slots	SetInventoryItems, SetEquipmentItems, SetBackpackItems
Coletas e inimigos fixos	Conjuntos de chaves de progresso	Desativação dos objetos correspondentes
Presentes de NPC e flechas	Bitmask, instante de reposição, drops pendentes	Regras de entrega e recriação de drops

Mudanças de slots, quantidade de pilhas, equipamentos e atributos disparam eventos que solicitam save. QueueSave espera pelo menos o quadro seguinte para capturar o estado após a mudança. PrologueChallenge também salva em checkpoints de 10 segundos de **tempo contado no mapa**. Portais, pausa, saídas e presentes fazem chamadas explícitas a SaveGame.

Ponto de atenção: a chave de progresso de um objeto fixo vem de WorldProgressKey.For, que combina caminho da cena, tipo de componente, nomes/índices da hierarquia e posição arredondada. Mover um item fixo, renomear seus pais ou mudar sua ordem na Hierarchy pode fazer um save antigo deixar de reconhecer aquele objeto. Slimes e curas gerados em runtime são separados dos objetos fixos.

4 Entrada e movimento

O arquivo autoral Player Controls.inputactions define os mapas Movement, Combat e Equipment. Player Controls.cs é código **gerado pelo Unity Input System**; alterações nele podem ser perdidas ao regenerar o asset. PlayerController usa a ação Movement.Move, com WASD, e recebe o Dash do mapa Combat. ActiveWeapon escuta o clique esquerdo pelo mapa Combat e cria uma InputAction para o clique direito. PowerController escuta Tab; BackpackController, B. Outros scripts leem teclas diretamente, como E em MenuController, F/Esc em NpcDialogueUI e Esc em OnionMenuController.

PlayerController.PlayerInput lê o vetor e atualiza os parâmetros moveX e moveY do Animator. Em FixedUpdate, Move chama Rigidbody2D.MovePosition. O personagem vira conforme a posição horizontal do cursor na tela. Dash multiplica a velocidade da cena por 4 por 0,2 segundo e liga um TrailRenderer; após 0,25 segundo adicional, aceita novo dash.

Ponto de atenção: a velocidade real de caminhada está serializada como 5 em cada cena, embora o valor inicial do campo C# seja 1. Para alterar o jogo atual, considere os valores do Inspector das duas cenas. Dash ainda não gasta estamina.

5 Atributos, dano e XP

PlayerVitals guarda atributos base de 100 vida, 5 defesa e 100 estamina; as cenas confirmam esses valores. EquipmentController.EquipmentChanged provoca RefreshBonuses, que soma healthBonus, defenseBonus e staminaBonus dos itens nos slots. StatsChanged atualiza o HUD, a aba Player e enfileira save. Vida e estamina são limitadas pelos respectivos máximos.

5.1 Fórmulas

Defesa base efetiva = $5 + (\text{nível} - 1)$

Defesa total = defesa base efetiva + bônus dos equipamentos

Dano recebido = $\text{máximo}(1, \text{dano bruto} - \text{defesa total})$

Dano da espada = dano do Item + $(\text{nível} - 1)$

XP necessária para passar do nível L ao L+1 = $100 + 50 \times (L - 1)$

TakeDamage ignora dano não positivo, player morto e dano que chegue durante a proteção de 0,5 segundo. Depois, subtrai a defesa, dispara flash, evento StatsChanged e, se a vida cair a zero, Died. UpdateAliveState desativa movimento e arma na morte. Heal não cura mortos nem consome cura quando a vida já está no máximo. GainExperience usa um laço: cada nível consome apenas sua exigência e mantém o excedente para o próximo.

Exemplo no nível 1	Sem armadura	Com capacete, peitoral e botas
Defesa	5	10
Ataque bruto de Fogo 12	7 de dano real	2 de dano real
Ataque bruto de Ghost 8	3 de dano real	1 de dano real

Ponto de atenção: PlayerVitals.BaseDefense inclui SwordDamageBonus, que na prática é o bônus de nível. O nome da propriedade pode surpreender quem a usa para UI, mas a soma corresponde à regra atual de +1 defesa por nível. A estamina é persistida e exibida; não há método atual de gasto/regeneração integrado ao dash ou ao arco.

6 Combate de espada e arco

6.1 Espada

ActiveWeapon.RefreshEquippedWeapons lê os slots Primário e Secundário. Ao equipar uma espada, instancia o weaponPrefab do item sob o objeto ActiveWeapon. As Espadas de Ferro e Diamante apontam atualmente para o prefab Iron_Sword; o sprite visível muda para o do item equipado, e o dano vem do campo weaponDamage do item. Sword.PerformAttack exige arma ativa, prefab de golpe e ponto de spawn; envia o gatilho Attack ao Animator, liga temporariamente o collider de ataque, cria o efeito de corte e inicia a recarga de 0,32 segundo. O evento da animação chama DoneAttackingAnimEvent para desligar o collider. Os estados SwingUp e SwingDown estão em velocidade 1,5 no Animator Controller.

DamageSource.OnTriggerEnter2D procura EnemyHealth no objeto atingido e chama TakeDamage com o player atacante. O inimigo soma XP ao atacante apenas quando a vida chega a zero. Sword.MouseFollowWithOffset orienta a arma para o cursor.

Ponto de atenção: o tempo de collider depende dos Animation Events existentes. Ao substituir clips de espada, mantenha ou recoloque DoneAttackingAnimEvent; sem ele, a área de acerto pode permanecer ativa até outro evento ou desativação.

6.2 Arco e flecha

Clique direito chama ActiveWeapon.FireArrow se o slot Secundário contém um arco e existe uma flecha no inventário ou na mochila. A rotina consome uma unidade, cria um GameObject de projétil com SpriteRenderer, Rigidbody2D e CircleCollider2D trigger, ignora colisões com o próprio player e usa ArrowProjectile.Launch. A velocidade é 12 unidades/segundo, duração máxima 4 segundos e dano 4. O sprite vem da referência configurada ou do item Flecha no dicionário. O arco visível aparece ao disparar ou enquanto o botão direito permanece pressionado; a espada é ocultada enquanto ele aparece. O arco e a espada são ajustados para a mesma Sorting Layer e ordem visual.

ArrowProjectile.OnTriggerEnter2D ignora o player dono, aplica dano ao primeiro inimigo atingido e destrói o projétil. O bônus de nível da espada não entra no dano da flecha.

7 Inimigos e geração dinâmica

EnemyAI decide entre patrulhar, perseguir e atacar. EnemyPathfinding, apesar do nome, faz **movimento direto por vetor** com Rigidbody2D.MovePosition; ele não calcula rota de navegação em torno de obstáculos. EnemyHealth controla vida, flash, recuo, barra e efeito de morte. EnemyHealthBar cria a UI acima do inimigo e reage a HealthChanged.

Prefab	Vida	Dano	XP	Visão	Desistência	Alcance de ataque
Fire_Slime	50	12	25	8	11	1,4
Ghost_Slime	25	8	15	8	11	1,4

SurvivalSpawner roda apenas na cena externa. O próximo slime aparece inicialmente a cada 3 segundos; o intervalo cai até 1 segundo conforme o progresso, respeitando 20 inimigos vivos. Ele escolhe um dos dois prefabs e sorteia variante normal, forte ou fraca. A chance de forte cresce de 20% para 40%; o limiar de fraco é roll > 0.7 quando o sorteio não caiu na faixa forte. Vida, dano, velocidade, escala e cor são ajustados logo após a instanciação.

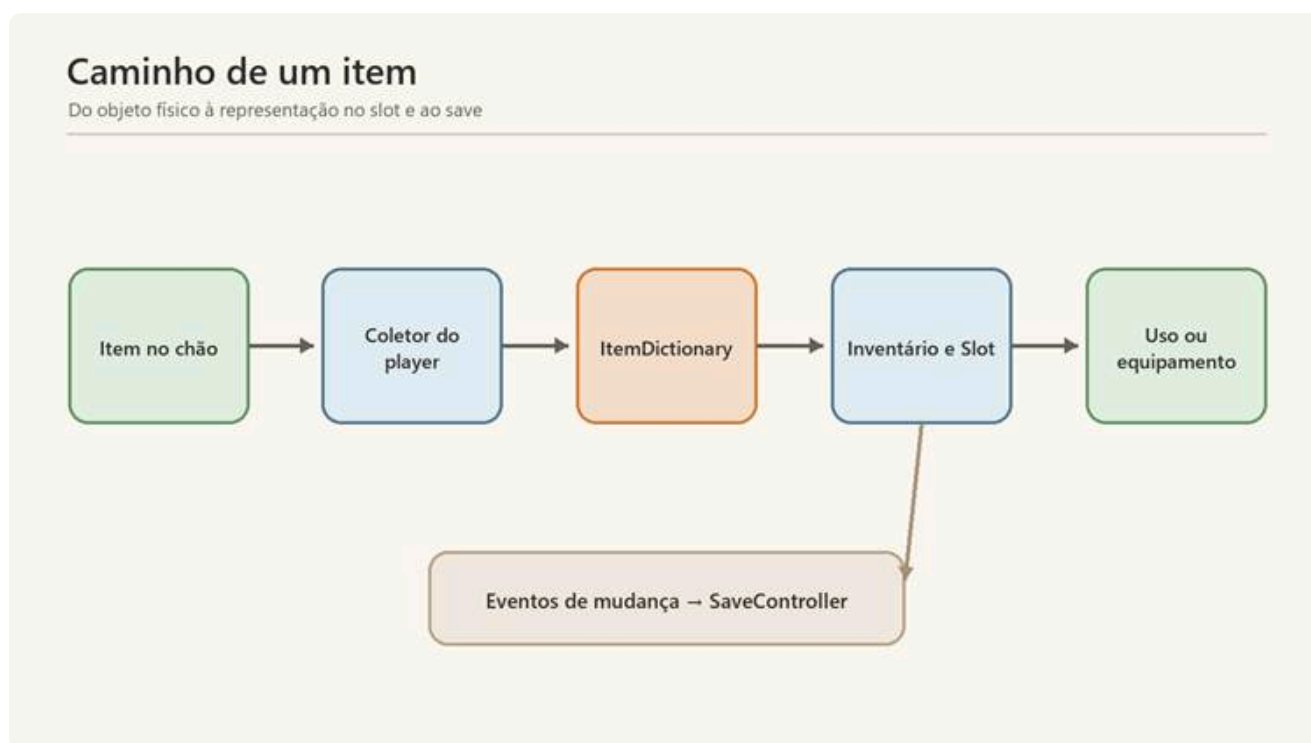
Para procurar posição, o spawner usa Grass.HasTile, exclui uma margem de três células do limite, consulta o Tilemap Foreground no centro e nos quatro vizinhos, exige que o slime esteja fora da câmera e rejeita colliders sólidos em um raio de 1 unidade. Ele tenta até 72 posições (36 configuradas vezes 2). O gerador de cura usa a mesma validação de terreno, procura área visível a 2,5–7 unidades do player e mantém no máximo 8 itens de cura gerados simultaneamente. Primeiro nascimento por volta de 4 segundos; depois, tentativas a cada 10 segundos.

Ponto de atenção: o gerador procura uma posição por tentativas. Com pouco terreno válido ou muitos obstáculos, uma tentativa pode não produzir objeto; a próxima acontece no intervalo seguinte. A visão, os limites de spawn e as referências dos prefabs são campos do Inspector da cena/prefab. A contagem de inimigos consulta objetos vivos por tipo; otimização pode ser futura caso o mapa cresça.

8 Modelo de item e inventário

Item reúne identificação (ID, Name), empilhamento, tipo de equipamento, tipo de poder, prefab de arma, dano e bônus de atributos ou cura. ItemDictionary recebe no Inspector a lista de prefabs canônicos. Em Awake, atribui IDs pela **ordem da lista, começando em 1**, e cria consultas por ID e nome. Coletar um item no mundo não move o GameObject físico para a UI:

InventoryController.AddItem encontra o prefab pelo nome e instancia uma representação no slot. CreateInventoryItem desliga SpriteRenderers e Colliders 2D do clone e centraliza sua Image.



Caminho de um item

PlayerItemCollector.OnTriggerEnter2D detecta a tag Item, tenta guardar a quantidade do objeto, mostra popup, concede 5 XP e destrói o objeto do mundo quando toda a quantidade foi aceita. ItemStack limita pilhas a 99 nos itens comuns elegíveis e a 1 nos equipamentos. O texto da quantidade é criado para pilhas maiores que 1. Slot representa um espaço comum; EquipmentSlot valida categorias e emite evento quando o equipamento muda. ItemDragHandler coordena arrastar, troca, combinação de pilhas, retorno ao slot original e consumo de cura por clique direito.

Regra	Onde procurar	Detalhe técnico
40 slots principais	InventoryController.slotCount nas cenas	SetInventoryItems cria/remove clones de slot até chegar à quantidade

Regra	Onde procurar	Detalhe técnico
Até 99 por pilha	Item.MaxStackSize, ItemStack	Equipamentos ficam limitados a 1
10 slots da mochila	BackpackSlotsPanel nas cenas	A janela é independente do Canvas do inventário
B só na aba Inventário	BackpackController.OnToggleBackpack	Exige mochila equipada e <code>CurrentTabIndex == 1</code>
Cura por clique direito	ItemDragHandler.OnPointerClick	Requer slot comum, <code>healAmount > 0</code> e vida abaixo do máximo
Item canônico	ItemDictionary	Busca por nome na coleta e por ID no carregamento

Ponto de atenção: como IDs dependem da ordem da lista de ItemDictionary, reordenar prefabs após criar saves pode restaurar itens errados. Para uma versão maior, use IDs fixos imutáveis e uma rotina de migração. Nomes também precisam ser únicos e consistentes com recompensas de NPC e munição.

9 Equipamentos, bônus e poderes

EquipmentSlotType enumera Primária, Escudo, Secundária, Poder1, Poder2, Mochila, Peitoral, Capacete e Botas. ItemEquipmentType informa o papel do item. ItemPowerType possui None, Absorbable e Contact. Um slot comum aceita qualquer Item; um slot de equipamento verifica compatibilidade. Poder1 e Poder2 aceitam apenas `equipmentType == Power` e `powerType != None`. Outros slots rejeitam itens que tenham um PowerType definido.

EquipmentController reúne os slots, emite EquipmentChanged, reconstrói os itens equipados durante o load e configura a regra de não retirar uma mochila com conteúdo.

PlayerVitals.RefreshBonuses percorre esses slots para somar atributos. PowerController acompanha qual dos dois poderes está selecionado. Tab só alterna quando os dois espaços contêm um item; se há apenas um, permanece nele. Um poder Absorvível é considerado travado (EquipmentSlot.IsLocked) e só pode ser removido por RemoveAbsorbablePower, que destrói o item.

Estado real da funcionalidade: há classificação, slots, seleção e bloqueio de Absorvíveis. Não há rotina que execute as 1–4 habilidades de cada item, aplique efeitos de Contato ou conecte uma ação de cenário à remoção do Absorvível. A interface ou o GDD futuro pode planejar isso, mas nenhum outro script deve assumir que a habilidade já existe.

10 NPCs, diálogo e presentes

NpcDialogue é um componente de dados: Role, nome mostrado, lista de falas e distância de interação. NpcDialogueUI.Update busca o NPC mais próximo dentro do raio, mostra dica de F e abre a UI. Ao abrir conversa, fecha inventário/mochila, desativa movimento e arma do player e mostra a primeira fala. F ou Espaço avança; Esc fecha. **O mundo não é pausado:** slimes e relógio continuam enquanto o diálogo está aberto. Se o player morrer, a conversa é fechada.

As quatro listas de falas estão vazias nas cenas, então RefreshLine mostra “Diálogo a definir.”. Ao terminar a conversa, GrantNpcReward consulta GameSession.ClaimedNpcRewards:

NPC	Regra de entrega	Persistência
Madrasta	Uma Espada de Ferro	Bit no mask de recompensas
Velho	Uma Espada de Diamante	Bit no mask de recompensas
Louco do Arco	Um Arco e flechas até 99; depois repõe até 99 após 60 s de tempo contado	Bit e próximo instante de reposição
Mercadora	Sem recompensa ou comércio codificados	Somente diálogo

Giveltent tenta primeiro o inventário; a capacidade restante vira PendingRewardDrop e é criada no chão. O save guarda o ID do drop, cena, nome, quantidade e posição. RefillArrows conta flechas no inventário, na mochila e nos drops ainda pendentes para não ultrapassar 99. O limite de 60 segundos usa GameSession.RunElapsedSeconds, que não aumenta na casa.

11 Relógio, pausa e telas de fim

PrologueChallenge existe nas duas cenas de jogo, mas countTimeInThisScene só é verdadeiro no mapa externo. Seu DurationSeconds depende da dificuldade: Fácil 180, Médio 600, Difícil 900. Insano não possui limite de vitória e exibe contagem crescente. Se o player está vivo, save carregado e o jogo não está pausado, Update soma Time.deltaTime. Ao alcançar o limite finito, marca GameSession.RunWon, salva e chama DeathScreenController.ShowVictory.

OnionMenuController gerencia menu inicial e pausa. No menu inicial, alterna sprites de fundo com WaitForSecondsRealtime e aplica blur. Em pausa, guarda o save, captura a tela no fim do quadro, aplica material de blur ao RawImage e ajusta Time.timeScale = 0. Esc ou botão retoma. DeathScreenController recebe o evento PlayerVitals.Died para exibir **DERROTADO** vermelho; para vitória, mostra **VITÓRIA** verde. Ele usa a mesma base visual e captura a cena antes de mostrar o painel. O botão da tela final chama RespawnFromBeginning e carrega a casa como nova tentativa.

Ponto de atenção: Time.timeScale deve voltar a 1 antes de carregar outra cena. Os controladores de pausa e de tela final fazem essa restauração nos caminhos de saída previstos. Ao criar novos botões de navegação, preserve essa regra.

12 UI e configurações

PlayerVitalsHUD reage a StatsChanged, preenche imagens de vida, estamina e XP e esconde o painel durante pausa, inventário, diálogo ou morte. PlayerStatsUI mostra nome, valores base e bônus na aba Player. EnemyHealthBar cria a barra sobre o inimigo. ItemPickupUIController gera popups de coleta. TabController ativa uma página por vez e emite TabChanged, usado pela mochila para fechar sua janela fora da aba Inventário.

DisplaySettingsPanel coloca opções 1920 × 1080/1280 × 720 e Tela cheia/Sem bordas nos dropdowns. ApplySelectedSettings escreve índices em PlayerPrefs; ApplySavedSettings reaplica ao abrir o menu. A configuração de vídeo não faz parte do JSON de cada save, pois vale para o jogo no computador.

SoundLibrary e SoundManager são os componentes de áudio presentes. Antes de especificar trilha e mixagem em design, verifique as referências de AudioSource e clips no Inspector; o projeto atual não expressa uma direção sonora final.

13 Catálogo completo dos scripts autorais

Os caminhos abaixo começam em Assets/Scripts/. Player Controls.cs é gerado pelo Input System e não faz parte da edição manual de comentários; seu arquivo fonte é Player Controls.inputactions.

13.1 Player

Arquivo	Papel
Player/PlayerController.cs	Lê movimento e dash, move Rigidbody2D, aponta o sprite para o cursor
Player/PlayerVitals.cs	Vida, estamina, defesa, XP, níveis, morte e eventos de status
Player/PlayerItemCollector.cs	Detecta item no chão, envia ao inventário, concede XP e registra coleta
Player/ActiveWeapon.cs	Equipa prefab de espada, alterna visual com arco, dispara flechas
Player/Equipamentos Player/EquipmentController.cs	Reúne slots equipados, notifica mudanças, salva/restaura equipamento
Player/Mochila/BackpackController.cs	Dez slots extras, regra da tecla B, capacidade e save da mochila
Player/Poderes Player/PowerController.cs	Seleção dos dois slots de poder e remoção de Absorvível

13.2 Itens e armas

Arquivo	Papel
Itens/Itens/Item.cs	Dados do item, tipo, dano, bônus, cura e identidade no mundo
Itens/Itens/ItemStack.cs	Quantidade, limite de pilha, combinação e etiqueta visual
Itens/ManipulationItem.cs	Classe ItemDragHandler: arrastar, trocar, juntar e consumir

Arquivo	Papel
Itens/Equipamentos Itens/EquipmentSlot.cs	Aceitação por tipo, bloqueio, retirada e evento de equipamento
Itens/Equipamentos Itens/EquipmentSlotType.cs	Enum dos destinos de equipamento
Itens/Poderes Itens/ItemPowerType.cs	Enum dos tipos None, Absorbable e Contact
Itens/Sword/Sword.cs	Ataque, animação, temporização, collider e orientação da espada
Itens/Sword/DamageSource.cs	Aplica dano de colisão de espada ao inimigo
Itens/Sword/ArrowProjectile.cs	Velocidade, colisão, dano e vida útil da flecha
Itens/Sword/SlashAnim.cs	Destrói o efeito de corte quando a partícula termina

13.3 Inventário, menus e UI

Arquivo	Papel
Menu/Inventário/Slot.cs	Interface ItemSlot e slot comum com evento de mudança
Menu/Inventário/InventoryController.cs	Adiciona, consome, conta, centraliza, salva e restaura itens
Menu/MenuController.cs	Tecla E abre ou fecha o inventário
Menu/GuiController.cs	Classe TabController: páginas e evento da aba ativa
Menu/OnionMenuController.cs	Menu principal, pausa, slideshow, blur e saída
Menu/SaveSlotPanelController.cs	Dificuldades, nome, cinco vagas, continuar e sobrescrever
Menu/DisplaySettingsPanel.cs	Resolução e modo de tela em PlayerPrefs
Menu/DeathScreenController.cs	Telas de vitória/derrota e reinício
UI/PlayerVitalsHUD.cs	Barras e textos do HUD do jogador
UI/PlayerStatsUI.cs	Status na aba Player
UI/EnemyHealthBar.cs	Barra de vida acima dos slimes
UI/ItemPickupUIController.cs	Popups com ícone/nome do item obtido

13.4 Mundo, inimigos e NPCs

Arquivo	Papel
Enemies/Criaturas/EnemyIA.cs	Classe EnemyAI: patrulha, visão, perseguição e ataque
Enemies/Criaturas/EnemyPathfinding.cs	Movimento direto do inimigo por Rigidbody2D
Enemies/Criaturas/EnemyHealth.cs	Vida, morte, XP, flash, recuo e barra

Arquivo	Papel
Enemies/Criaturas/SurvivalSpawner.cs	Nascimento aleatório de slimes e cura com validação de terreno
NPCs/NpcDialogue.cs	Papel, nome, falas e alcance de interação do NPC
NPCs/NpcDialogueUI.cs	Conversa, navegação de falas e recompensas
Misc/ScenePortal.cs	Salva e muda de cena ao atravessar porta
Misc/SceneSpawnPoint.cs	Identifica pontos de nascimento e chegada
Misc/CameraFollow.cs	Centraliza e segue o jogador
Misc/Flash.cs	Piscar visual ao levar dano
Misc/Knockback.cs	Recuo físico após impacto

13.5 Partida, persistência e áudio

Arquivo	Papel
Misc/GameSession.cs	Estado transitório entre cenas e dificuldade da tentativa
Misc/PrologueChallenge.cs	Cronômetro, vitória e checkpoints de autosave
Misc/SaveData.cs	Estruturas serializáveis do JSON, equipamentos e drops penderes
Misc/InventorySaveData.cs	ID, posição e quantidade de um item em slot
Misc/SaveController.cs	Orquestra save/load, progresso do mundo e recompensas
Misc/SaveSlotService.cs	Nomes de arquivo e validação das cinco vagas
Misc/ItemDictionary.cs	Mapeia nomes/IDs para os prefabs canônicos
Misc/WorldProgressKey.cs	Identidade estável de item ou inimigo fixo na cena
MusicAudio/SoundLibrary.cs	Referências de clips/recursos sonoros
MusicAudio/SoundManager.cs	Reprodução de áudio do jogo

14 Referências do Inspector que devem acompanhar mudanças

Mudança desejada	Arquivo ou objeto a revisar	Por quê
Novo item coletável	Prefab Item, ItemDictionary das duas cenas, SpriteRenderer, Image, tag Item, Collider2D	Coleta por nome e load por ID exigem prefab canônico
Nova espada	Item com PrimaryWeapon e weaponPrefab, prefab com Sword, collider e DamageSource	O visual, a animação e o dano são partes diferentes

Mudança desejada	Arquivo ou objeto a revisar	Por quê
Novo NPC	Objeto com NpcDialogue, role, nome, falas, collider/posição	A UI busca NPCs por distância; a recompensa depende do role no código
Nova cena	Build Settings, portais e SceneSpawnPoint, player, UI, SaveController	Transição e save precisam de objetos equivalentes
Novo inimigo	Prefab com EnemyHealth, EnemyAI, EnemyPathfinding, Collider2D, Rigidbody2D	Spawn, ataque e morte dependem desses componentes
Nova cura	Prefab Item com healAmount > 0, dicionário e lista do spawner	O clique direito e o gerador usam os dados do item
Balancear timer	GameSession.Difficulty, PrologueChallenge	Valor escolhido persiste em save
Ajustar controles	Player Controls.inputactions ou scripts que usam Input direto	O arquivo C# gerado não é fonte de edição

As referências de UI e prefabs diferem por cena. Alterar uma cena apenas pode deixar a outra com comportamento diferente. No caso da mochila e do inventário, os slots dos dois locais precisam estar presentes nas cenas de jogo; seu painel pode começar desativado, porque o load e o código buscam objetos inativos quando necessário.

15 Roteiros de estudo

15.1 Do chão até a cura

Abra Item.cs e observe Name, MaxStackSize e healAmount. Depois siga
 PlayerItemCollector.OnTriggerEnter2D → InventoryController.AddItemQuantity →
 ItemDictionary.GetItemPrefab(string) → InventoryController.CreateInventoryItem →
 Slot.TrySetItem. Para consumir, siga ItemDragHandler.OnPointerClick → PlayerVitals.Heal →
 ItemStack.RemoveUpTo ou destruição do clone. Por fim, veja como AnySlotChanged e
 AnyQuantityChanged chamam SaveController.QueueSave.

15.2 Do clique até a morte do slime

Siga ActiveWeapon.OnPrimaryAttack → Sword.PerformAttack →
 DamageSource.OnTriggerEnter2D → EnemyHealth.TakeDamage → PlayerVitals.GainExperience
 → EnemyHealth.DetectDeath. Para o arco, troque os passos da espada por OnSecondaryAttack
 → FireArrow → TryConsumeArrow → ArrowProjectile.Launch → OnTriggerEnter2D.

15.3 Da escolha de dificuldade à vitória

Siga SaveSlotPanelController.OpenNewGame → escolha de dificuldade →
 GameSession.ChooseNewGame → SaveController.ResetProgress → ScenePortal →
 PrologueChallenge.Update → GameSession.MarkRunWon →

DeathScreenController.ShowVictory. Compare countTimeInThisScene nas duas cenas e a condição HasTimeLimit em Insano.

15.4 Do equipamento ao bônus de defesa

Arraste um Item de armadura até EquipmentSlot. O slot valida ItemEquipmentType, emite ItemChanged, EquipmentController emite EquipmentChanged e PlayerVitals.RefreshBonuses soma o valor. PlayerStatsUI.Refresh mostra base e acréscimo; SaveController grava o ID no registro de equipamento. No load, o dicionário reinstancia o item no mesmo slot.

16 Pontos de manutenção e limites conhecidos

7. **Classes geradas.** Edite Player Controls.inputactions; o arquivo Player Controls.cs é gerado.
8. **IDs por ordem.** Preserve a ordem dos prefabs do ItemDictionary ou planeje migração de saves.
9. **Chaves de objetos fixos.** Evite renomear/reordenar/mover objetos do mundo sem aceitar o impacto em saves antigos.
10. **Referências de cena.** Portais, spawn points, UI, dicionário e spawner dependem de configurações de Inspector.
11. **IA direta.** EnemyPathfinding não contorna obstáculos por busca de caminhos; slimes podem ficar presos em geometrias complexas.
12. **Mistura de entrada.** O projeto usa tanto o novo Input System quanto leituras diretas de teclado; uma futura tela de remapeamento terá que unificar esses pontos.
13. **Poderes parciais.** Os tipos e slots existem, mas habilidades, custos e ações de retirada ainda pedem implementação.
14. **Mercadora parcial.** A personagem conversa, mas não há compra/venda, moeda funcional ou preços.
15. **Falas provisórias.** As listas dialogueLines estão vazias nas cenas; a narrativa precisa ser preenchida no Inspector.
16. **Spawner por tentativa.** Ao diminuir a área livre do mapa, pode ser necessário adaptar distância, margem, checagem de câmera e número de tentativas.
17. **Escalabilidade.** Consultas frequentes com FindObjectsByType e instanciação de UI/projéteis são aceitáveis no protótipo, mas merecem perfilamento antes de mapas maiores.
18. **Sem verificação de Play neste documento.** Os fluxos foram reconstruídos pelos scripts e referências serializadas. A experiência visual e os valores em execução devem ser conferidos no Editor depois de futuras mudanças.

17 Glossário para estudo

Termo	Significado neste projeto
Prefab	Modelo reutilizável que a cena instancia para item, arma, inimigo ou efeito
Scene	Arquivo Unity com objetos, posição, hierarquia e referências
MonoBehaviour	Classe C# anexada a GameObject, com ciclo Awake/Start/Update
Collider trigger	Área que dispara evento de contato sem bloquear fisicamente
Rigidbody2D	Componente da física 2D usado para deslocamento/colisão
Tilemap	Grade de tiles do mapa; Grass representa chão, Foreground representa obstáculos
Serialized field	Campo mostrado/salvo no Inspector, que pode prevalecer sobre o valor inicial do C#
Event	Notificação C# que atualiza UI ou save após uma mudança
SaveData	Estrutura serializável convertida para JSON
Runtime spawn	Objeto criado durante a partida, diferente de um objeto fixo da cena
Sorting Layer	Ordem de desenho de sprites no mundo